Code Explanation

SQL Validation Queries for Sales Orders

This SQL script is designed to validate the data quality and consistency of sales orders data across staging and main tables. It checks for various conditions such as row counts, null values, data consistency, and data quality.

Overview of the Code
The given SQL script contains a series of validation queries for sales orders data. It checks the row count, null values, and data consistency between the staging and main tables. Additionally, it verifies the data quality in the main table by checking for null values, date fields validity, numeric calculations, and data filtering by year.

Step 1: Checking Row Count in Staging Table
This step retrieves the total row count from the `sales_orders_comp_stg` table to ensure that data is present and to get an initial count for comparison.

```
%sql
SELECT COUNT(*) AS row_count_stg
FROM sales_orders_comp_stg;
```

Step 2: Checking for Null Values in Key Fields of Staging Table
This step checks for null values in key fields (`CompOrderNumber`, `CompLineNumber`, `CompMaterialNumber`) of the `sales_orders_comp_stg` table to identify any missing data.

```
%sql
SELECT
    SUM(CASE WHEN CompOrderNumber IS NULL THEN 1 ELSE 0 END) AS null
_order_number,
    SUM(CASE WHEN CompLineNumber IS NULL THEN 1 ELSE 0 END) AS null_
line_number,
    SUM(CASE WHEN CompMaterialNumber IS NULL THEN 1 ELSE 0 END) AS n
ull_material_number
FROM sales_orders_comp_stg;
```

Step 3: Checking Row Count in Main Table
This step retrieves the total row count from the `sales_orders_comp` table to compare with the staging table's row count.

```
%sql
SELECT COUNT(*) AS row_count_main
FROM sales_orders_comp;
```

Step 4: Checking Data Consistency Between Staging and Main Tables
This step performs a left join between `sales_orders_comp_stg` and `sales_orders_comp` tables on key fields to identify records present in the staging table but missing in the main table.

```
%sql
SELECT
    stg.CompOrderNumber,
    stg.CompLineNumber,
    stg.CompMaterialNumber,
    main.CompOrderNumber AS main_order_number,
    main.CompLineNumber AS main_line_number,
    main.CompMaterialNumber AS main_material_number,
    CASE
        WHEN main.CompOrderNumber IS NULL THEN 'Missing in main'
        ELSE 'Present in both'
    END AS status
FROM
    sales_orders_comp_stg stg
LEFT JOIN
    sales_orders_comp main
ON
    stg.CompOrderNumber = main.CompOrderNumber
    AND stg.CompLineNumber = main.CompLineNumber
WHERE
    main.CompOrderNumber IS NULL
LIMIT 100;
```

Step 5: Checking Data Quality in Main Table

This step checks for null values in key fields (`CompOrderNumber`, `CompLineNumber`, `CompMaterialNumber`, `CompSiteId`, `CompShipToNumber`, `CompSoldToNumber`) of the `sales_orders_comp` table.

```
%sql
SELECT
    SUM(CASE WHEN CompOrderNumber IS NULL THEN 1 ELSE 0 END) AS null
_order_number,
    SUM(CASE WHEN CompLineNumber IS NULL THEN 1 ELSE 0 END) AS null_
line_number,
    SUM(CASE WHEN CompMaterialNumber IS NULL THEN 1 ELSE 0 END) AS n
ull_material_number,
    SUM(CASE WHEN CompSiteId IS NULL THEN 1 ELSE 0 END) AS null_site
_id,
    SUM(CASE WHEN CompShipToNumber IS NULL THEN 1 ELSE 0 END) AS nul
l_ship_to_number,
    SUM(CASE WHEN CompSoldToNumber IS NULL THEN 1 ELSE 0 END) AS nul
l_sold_to_number
FROM
    sales_orders_comp;
```

Step 6: Checking Date Fields Validity

This step checks the validity of date fields (`CompCreateDate`, `CompPromisedDeliveryDate`, `CompRequestedDeliveryDate`) in the `sales_orders_comp` table.

```
%sql
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN CompCreateDate > CURRENT_DATE() THEN 1 ELSE 0 END)
 AS future_create_dates,
```

```
    SUM(CASE WHEN CompPromisedDeliveryDate < CompCreateDate THEN 1 E
LSE 0 END) AS invalid_promise_dates,
    SUM(CASE WHEN CompRequestedDeliveryDate < CompCreateDate THEN 1
ELSE 0 END) AS invalid_request_dates
FROM
    sales_orders_comp;
```

Step 7: Checking Numeric Calculations

This step verifies the consistency of numeric calculations (`CompOpenQuantityOrginal`, `CompOrderQuantityOriginal`, `CompCancelledQuantityOriginal`, `CompDeliveredQuantityOriginal`) in the `sales_orders_comp` table.

```
%sql
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN CompOpenQuantityOrginal != (CompOrderQuantityOrigi
nal - CompCancelledQuantityOriginal - CompDeliveredQuantityOriginal)
 THEN 1 ELSE 0 END) AS inconsistent_quantities
FROM
    sales_orders_comp;
```

Step 8: Verifying Data Filtering by Year

This step checks if the data is filtered correctly by year based on the `CompCreateDate` field.

```
%sql
SELECT
    YEAR(CompCreateDate) AS order_year,
    COUNT(*) AS order_count
FROM
    sales_orders_comp
GROUP BY
    YEAR(CompCreateDate)
ORDER BY
    order_year;
```